

FIRST ATTEMPT:

```
#DEFINE N5
```

```
Void philosopher (int i)
```

```
{
```

```
While true
```

```
{
```

```
Think (); // for some time
```

```
Take_fork (Ri)
```

```
Take_fork (Li)
```

```
Eat ();
```

```
Put_fork (Li)
```

```
Put_fork (Ri)
```

```
}
```

```
}
```

SECOND ATTEMPT:

```
#DEFINE N5
```

```
Void philosopher (int i)
```

```
{
```

```
While true
```

```
{
```

```
Think ();
```

```
Take_fork (Ri)
```

```
If (available (Li)
```

```
{
```

```
Take_fork (Li)
```

```
Eat ();
```

```
Put_fork (Ri)
```

```
Put_fork (Li)
```

```
}
```

```
Else
{
Put_fork (Ri)
Sleep (T)
}
}
```

In the solution two instead of sleep we can change to sleep (random_time), still is not guarantee that the starvation will not occur.

THIRD ATTEMPT:

```
#DEFINE N5

Void philosopher (int i)
{
While true
{
Think ();
Lock (mutex)
Take_fork (Ri)
Take_fork (Li)
Eat ();
Put_fork (Li)
Put_fork (Ri)
Unlock (mutex)
}
}
```

FOURTH ATTEMPT USING SEMAPHORES

```
Void philosopher (int i)                void take_forks (int i)                void put_forks (int i)
{
While (TRUE)
{
Think ();
Take_forks (i);
Eat ();
Put_forks ();
}
}

void take_forks (int i)
{
lock (mutex);
state [i] = HUNGRY
test (i);
unlock (mutex)
down (s[i])
}

void put_forks (int i)
lock(mutex)
state [i] = THINKING
test (LEFT)
test (RIGHT)
unlock (mutex);
}

Void test (int i)
{
If (state [i] = HUNGRY && state [LEFT]! = EATING && state [RIGHT]! = EATING)
{
State [i] = EATING
UP (S[i]);
}
}
```